

# Docker user-defined Networks

*Virtualizzazione e integrazione di sistemi*

*Vittorio Ghini*

*a.a. 2022-23*

*Ingegneria e scienze informatiche*

*Università di Bologna*

# Contents

- 1. Useful Docker command
- 2. Work with Network command
  - 2.1 Create user-defined bridge networks
  - 2.2 Create container attached to a network
  - 2.3 Attach container to a network
  - 2.4 Docker Embedded DNS server

# 1.1. Docker **inspect** command

- Return low-level information on Docker objects.

```
docker inspect [OPTIONS] NAME|ID [NAME|ID...]
```

- By default, docker inspect command renders results in a JSON array.
- You can specify **format** argument (using Go syntax) to show selected parts only.

Start a container and try:

```
docker run --name=myubuntu ubuntu
```

```
docker inspect myubuntu
```

```
[  
  {  
    "Id": "daad4ae20ce419e51a745b2a30db76b4fea7de1191e9e29582f2190d3c2e923c",  
    "Path": "/bin/bash",  
    "State": { "Status": "exited", "Running": false, "ExitCode": 0 },  
    "HostConfig": { "Binds": null, },  
    "NetworkMode": "default",  
    "NetworkSettings": { "Bridge": "", "Ports": {}, "IPAddress": "",  
      "Networks": {  
        "bridge": { "IPAddress": "", "MacAddress": "" }  
      }  
    }  
  }  
]
```

# 1.1. Docker **inspect** command

- <https://docs.docker.com/engine/reference/commandline/inspect/>

Examples

**Get a result in json format**

**Networks**

```
docker inspect --format='{{json .NetworkSettings.Networks}}' CONTAINER_ID
```

**Ippaddress of the bridge network**

```
docker inspect --format='{{json .NetworkSettings.Networks.bridge.IPAddress}}' CONTAINER_ID
```

# 1.1. Docker inspect command

- <https://docs.docker.com/engine/reference/commandline/inspect/>

## Examples

### Get a container's IP address

```
docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'  
CONTAINER_ID
```

### Get an container's MAC address

```
docker inspect --format='{{range .NetworkSettings.Networks}}{{.MacAddress}}{{end}}'  
CONTAINER_ID
```

### List all port bindings

You can loop over arrays and maps in the results to produce simple text output:

```
docker inspect --format='{{range $p, $conf := .NetworkSettings.Ports}} {{$p}} ->  
{{(index $conf 0).HostPort}} {{end}}' CONTAINER_ID
```

### Get an container's image name

```
docker inspect --format='{{.Config.Image}}' CONTAINER_ID
```

# 1.2. Docker network inspect command

- Return low-level information on Docker objects.

```
docker network inspect [OPTIONS] NETWORK_NAME|NETWORK_ID
```

You can specify **format** argument (using Go syntax) to show selected parts only.

```
docker network inspect bridge
```

```
[
  {
    "Name": "bridge", "Scope": "local",
    "IPAM": {
      "Config": [ { "Subnet": "172.17.0.0/16" } ]
    },
    "Internal": false,
    "Containers": {
      "f48fc7c2f0993ef49a0ad157717b999a11300699728ad13214d56554b387133c": {
        "Name": "hello2_web_1",
        "EndpointID": "08993ebb840f18996ca494795f457751b3117e7517e025b557c289fcbd241732",
        "MacAddress": "02:42:ac:11:00:03",
        "IPv4Address": "172.17.0.3/16",
        "IPv6Address": ""
      }
    },
    "Options": {
      "com.docker.network.bridge.default_bridge": "true",
      "com.docker.network.bridge.enable_icc": "true",
      "com.docker.network.bridge.enable_ip_masquerade": "true",
      "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
      "com.docker.network.bridge.name": "docker0",
      "com.docker.network.driver.mtu": "1500"
    }
  },
]
```

# 1.2. Docker network inspect command

```
docker network inspect --format='{{json .Options}}' bridge
```

```
"Options": {  
    "com.docker.network.bridge.default_bridge": "true",  
    "com.docker.network.bridge.enable_icc": "true",  
    "com.docker.network.bridge.enable_ip_masquerade": "true",  
    "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",  
    "com.docker.network.bridge.name": "docker0",  
    "com.docker.network.driver.mtu": "1500"  
},  
}  
]
```

## 2. Work with Network command

These are the network subcommands you can use to interact with Docker networks and the containers in them.

- `docker network create`
- `docker network connect`
- `docker network ls`
- `docker network rm`
- `docker network disconnect`
- `docker network inspect`

- References:

<https://docs.docker.com/v17.09/engine/userguide/networking/work-with-networks/>

<https://docs.docker.com/v17.09/engine/userguide/networking/>



## 2.1 Create user-defined bridge networks

- Docker Engine creates a bridge network automatically when you install Engine. This network corresponds to the docker0 bridge that Docker Engine has traditionally relied on. In addition to this network, you can create your own bridge or overlay network.
- A bridge network resides on a single host running an instance of Docker Engine. Instead, an overlay network can span multiple hosts running their own engines.
- If you run **docker network create** and supply only a **network name**, it creates a bridge network for you.

```
docker network create simple-network
```

```
docker network create -d bridge simple-network
```

The -d option specify the driver which manages the network, the default is "bridge".

You can run the following command to show the network configuration.

```
docker network inspect simple-network
```

## 2.1 Create user-defined bridge networks

You can run the following command to show the network configuration.

```
docker network inspect simple-network
```

You can run the following command to show the available networks on your host.

```
docker network ls
```

| NETWORK ID   | NAME           | DRIVER | SCOPE |
|--------------|----------------|--------|-------|
| 2a8106595d26 | bridge         | bridge | local |
| 139e647b51ad | host           | host   | local |
| 44916b258055 | none           | null   | local |
| fd8e65e19c60 | simple-network | bridge | local |

You can run the following command to delete a given network.

```
docker network rm NETWORK_NAME | NETWORK_ID
```

```
docker network rm simple-network
```

## 2.1 Create user-defined bridge networks

- **Note:** It is highly recommended to use the `--subnet` option while creating a network. If the `--subnet` is not specified, the docker daemon automatically chooses and assigns a subnet for the network and it could overlap with another subnet in your infrastructure that is not managed by docker. Such overlaps can cause connectivity issues or failures when containers are connected to that network.
- Only overlay networks support multiple subnets.
- In addition to the `--subnet` option, you also specify the `--gateway` and several `--aux-address` options.

```
docker network create --subnet=192.168.0.0/16 --gateway=192.168.0.100 --aux-  
address="my-router=192.168.1.3" --aux-address="my-switch=192.168.1.4" --aux-  
address="my-printer=192.168.1.5" --aux-address="my-nas=192.168.1.6" -d  
bridge my-network
```

# 2.1 Create user-defined bridge network

## Network create options for bridge network driver:

| Option                                                                                    | Equivalent       |
|-------------------------------------------------------------------------------------------|------------------|
| • com.docker.network.bridge.name<br>bridge name to be used when creating the Linux bridge |                  |
| • com.docker.network.bridge.enable_ip_masquerade<br>Enable IP masquerading                | <b>--ip-masq</b> |
| • com.docker.network.bridge.enable_icc<br>Enable or disable inter container connectivity  | <b>--icc</b>     |
| • com.docker.network.bridge.host_binding_ipv4<br>Default IP when binding container ports  | <b>--ip</b>      |
| • com.docker.network.driver.mtu<br>Set the containers network MTU                         | <b>--mtu</b>     |

## Network create options for any network driver:

|                   |                                              |
|-------------------|----------------------------------------------|
| <b>--internal</b> | Restrict external access to the network      |
| <b>--ipv6</b>     | Enable IPv6 networking for any network drive |

## 2.2 Create container attached to a network

- The following example uses `-o` to bind to a specific IP address available on the host when binding ports, then uses `docker network inspect` to inspect the network, and finally **attaches a new container to the new network**.
- Note that you should replace the IP address `10.0.2.15` shown in the example with an IP address available on a network interface in your host.
- Moreover, the `-P` option publish all exposed ports to random ports on host

```
docker network create -o
```

```
"com.docker.network.bridge.host_binding_ipv4"="10.0.2.15" my-network
```

```
docker network inspect my-network
```

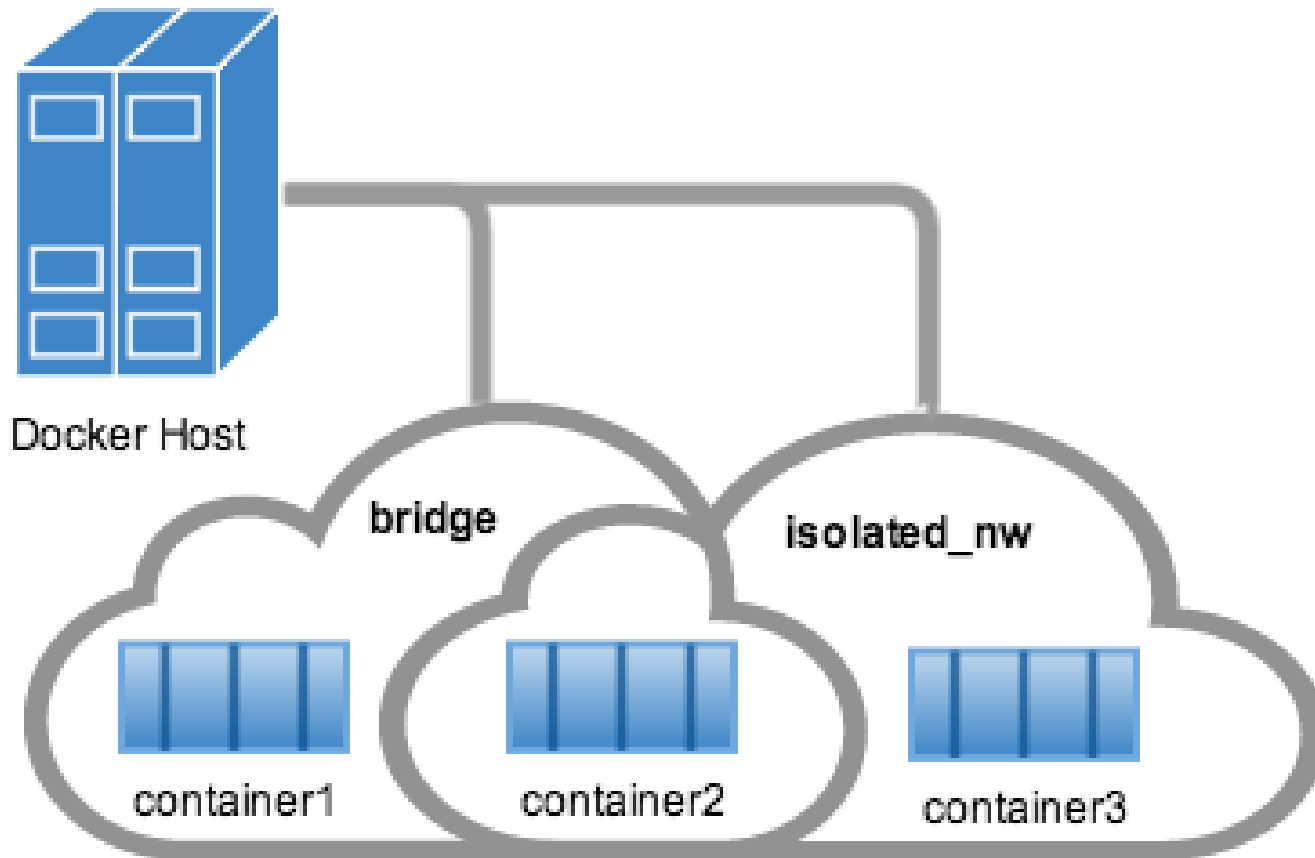
```
docker run -d -P --name redis --network my-network redis
```

```
docker ps
```

| CONTAINER ID | IMAGE | COMMAND                  | CREATED | STATUS            | PORTS                     | NAMES    |
|--------------|-------|--------------------------|---------|-------------------|---------------------------|----------|
| 476a145ab6f1 | redis | "docker-entrypoint.s..." | ...     | Up About a minute | 10.0.2.15:32770->6379/tcp | redis 13 |

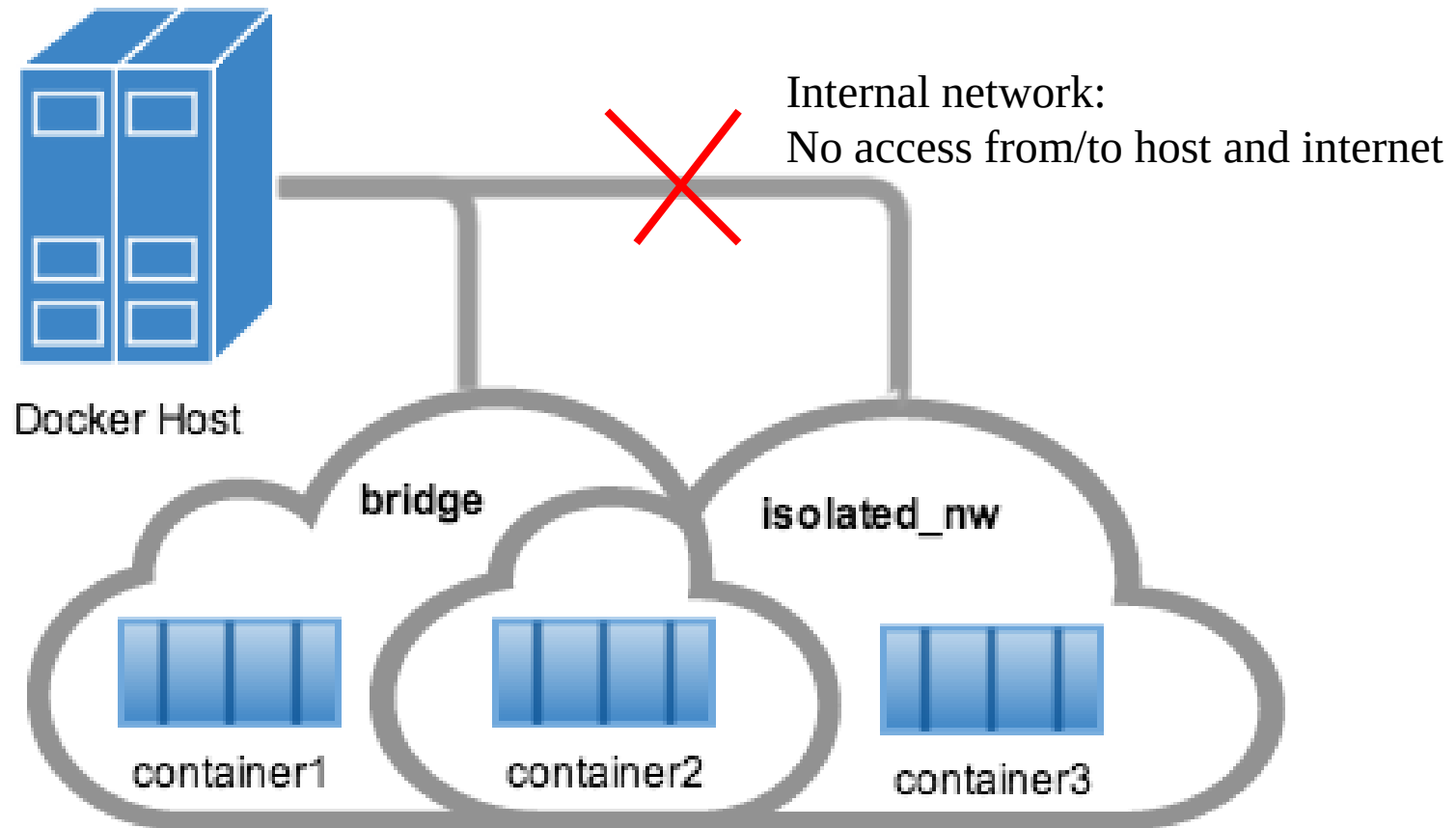
## 2.3 Attach container to a network

- You can connect an existing container to one or more networks.
- A container can connect to networks which use different network drivers.
- Once connected, the containers can communicate using another **container's IP address** or **name**.
- The example



## 2.3 Attach container to a network

- container3 is connected to the isolated\_nw network only
- container1 is connected to the default bridge network only
- container2 is connected to both the isolated\_nw and default bridge networks.



## 2.3 Attach container to a network

- Create and run two containers, container1 and container2

```
docker run -itd --name=container1 busybox
docker run -itd --name=container2 busybox
```
- Create an isolated, bridge network to test with

```
docker network create -d bridge --internal --subnet 10.2.2.0/24
isolated_nw
```

- Connect container2 to the isolated\_nw network:

```
docker network connect isolated_nw container2
```

Inspect the isolated\_nw network to verify the connection:

- ```
docker network inspect isolated_nw
```
- Notice that **container2** is assigned an IP address **automatically**. Because you specified a --subnet when creating the network, the IP address was chosen from that subnet.
- As a reminder, container1 is only connected to the default bridge network.



## 2.3 Attach container to a network

- Start a third container, but this time assign it an IP address using the `--ip` flag and connect it to the `isolated_nw` network using the `docker run` command's `--network` option

```
docker run --network=isolated_nw --ip=10.2.2.3 -itd --name=container3  
busybox
```

- Inspect the network resources used by **container3** and notice that the `container3` is not connected to the default bridge network.

```
docker inspect container3
```

- Inspect the network resources used by `container2`

```
docker inspect container2
```

- Notice that `container2` belongs to two networks (`eth0` and `eth1`). It joined the default bridge network when you launched it and you connected it to the `isolated_nw`

## 2.3 Attach container to a network

- Use the docker attach command to connect to the running container2 and examine its networking stack:

```
docker attach container2
```

- Now you are in the shell of container2. Use the ifconfig command to examine the container's networking stack. You should see two ethernet interfaces, one for the default bridge network, and the other for the isolated\_nw network.

```
#/ ifconfig -a
```

Detach from container2 and leave it running using CTRL-p CTRL-q.

## 2.3 Attach container to a network

- As long as the IP address you specify for the container is part of the network's subnet, you can assign an IPv4 or IPv6 address to a container when connecting it to a network, by using the `--ip` or `--ip6` flag.
- When you specify **an IP address** in this way **while using a user-defined network**, the configuration is preserved as part of the container's configuration and will be applied when the container is reloaded (after saving image).
- **Assigned IP addresses are not preserved** when using **non-user-defined networks**, because there is no guarantee that a container's subnet will not change when the Docker daemon restarts unless you use user-defined networks

## 2.4 Docker Embedded DNS server

- The Docker embedded DNS server enables name resolution for containers connected to a given network.
- This means that **any connected container can ping another container on the same network by its container name.**
- In the example, from within container2, you can ping container3 by name.
- After started the "docker attach container2", we are in the running shell of container2.
- From container2, try to ping container3 by name. Ok.  
ping container3
- Again, from container2, try to ping container1 by name. Ok  
ping container1
- Detach from container2 and leave it running using CTRL-p CTRL-q.

## 2.4 Docker Embedded DNS server

- `ERROR`, from container3, try to ping container1 by name. Unreachable.  
`docker attach container3`
- After started the "docker attach container3", we are in the running shell of container3.
- From container3, try to ping container1 by name. Unreachable.  
`ping container1`